

Claude Code Mastery

A 16-Module Condensed Study Guide

Author: Srinivas Vara

Modules: CCo — CC15 (16 modules)

Format: Condensed reference — Big Idea, Analogy, How It Works,
Pseudocode, Misconceptions, Key Takeaway, Quick Quiz

Table of Contents

CC0 Getting Started with Claude Code

Master the mental model, install the CLI, and run your first session.

CC1 API Essentials for CLI Power Users

Master models, Messages API, system prompts, temperature, and structured output.

CC2 Prompt Engineering for Claude Code

Five techniques to make Claude do the right thing reliably: clarity, specificity, XML structure, examples, thinking.

CC3 CLAUDE.md & Memory

How Claude Code remembers things across sessions, repos, and machines via configuration cascades and memory shortcuts.

CC4 Permissions & Sandbox

Two independent safety layers: ask for permission, enforce boundaries with OS-level jail.

CC5 Skills & Slash Commands

Build custom commands as unified skills; use SKILL.md, shell injection, and fork execution.

CC6 Subagents

Specialist agents in isolated contexts—delegate verbose work, keep your conversation clean and focused.

CC7 Hooks

Programmatic control over tool calls. Auto-format, block, route policy, inject context.

CC8 Tool Use Deep Dive

Master the four-step tool-use loop, multi-tool orchestration, and fine-grained control.

CC9 MCP — Model Context Protocol

Wire GitHub, Postgres, Slack, and custom tools into Claude Code as first-class capabilities using the open MCP protocol.

CC10 Claude Features — Thinking, Caching, Citations, Files

Extended reasoning, cost-cutting caching, document input, citations, and sandboxed code execution

CC11 Evaluating Skills, Subagents & Prompts

Treat your skills and prompts like code: test them, measure regressions, gate merges.

CC12 RAG for Claude Code

Retrieval, embeddings, BM25, hybrid pipelines—and how to wrap RAG as a tool.

CC13 Workflows, Agents & Computer Use

Four canonical patterns for orchestrating multi-step Claude work plus computer use for GUI automation.

CC14 Power User & CI/CD

Master extended thinking, parallel sessions, and Claude-driven code review in production pipelines.

CC15 Building Custom Agents with the Claude Agent SDK

Package Claude Code's agent engine as a Python and TypeScript library for background jobs, scheduled tasks, and programmatic automation.

CCo: Getting Started with Claude Code

Master the mental model, install the CLI, and run your first session.

The Big Idea

Claude Code is not a chat plugin or autocomplete tool—it's a command-line orchestration framework that runs in your terminal and executes real work on your codebase using built-in tools (Read, Write, Edit, Bash, Glob, Grep) plus extensible integrations via MCP servers, skills, subagents, and hooks. It edits your filesystem with your permission, runs commands like `npm test` and `git diff`, and remembers context across sessions. This transforms Claude from a stateless answering machine into a persistent, tool-using teammate working directly in your project.

Analogy

Think of the difference between a vending machine and a kitchen with a chef. A vending machine takes a coin and dispenses one snack—each turn is isolated. ChatGPT in a browser is a vending machine: you ask a question, you get text you copy-paste. Claude Code, by contrast, is a kitchen with a chef who has read and execute access to your codebase, can run your build tools, remember what happened yesterday, and propose edits you can review and merge. It operates inside your project, not outside narrating recipes.

How It Works

1. Install the CLI via npm and authenticate with your account (Pro/Team/Enterprise via OAuth, or API key via Anthropic Console).
2. Open Claude Code in a terminal inside a git repository or any writable directory.
3. Write a natural-language request: "Add JWT auth to the API" or "Refactor this function to use async/await."
4. Claude Code reads relevant files, searches the codebase, and proposes edits with diffs you review before accepting.
5. Use commands like `/init`, `/help`, `/status`, `/plan`, `/model`, `/clear`, `/compact`, `/agents`, `/hooks` to configure behavior and manage sessions.
6. Reference files with `@`, switch into plan mode (think-first, edit-later), and resume yesterday's session with `--continue` or `--resume`.

Pseudocode

When user opens Claude Code in a terminal:

1. Authenticate via OAuth (Pro/Team) or API key (Console)
2. Read repository metadata and optional CLAUDE.md config
3. Listen for natural-language task prompt
4. Parse request and identify tools needed (Read, Grep, Bash, Glob)
5. Execute reads and searches WITHOUT permission per file
6. Propose edits as diffs
7. Wait for user approval before writing filesystem
8. If edit approved: write file, report results
9. If denied: propose alternative approach
10. Save session state so --continue or --resume works
11. Close session OR receive next prompt (multi-turn)
12. Persist context in CLAUDE.md for future sessions

Common Misconceptions

- **Myth:** Claude Code is Cursor or Copilot in VS Code. **Reality:** It's a terminal CLI that runs independently; it has an optional IDE extension but is not an inline editor. It reads and edits files, runs commands, and remembers state across terminal sessions.
- **Myth:** I should just tell it "write the code" and paste the result. **Reality:** Plan mode (Shift+Tab) is the right first move for risky or unfamiliar work. Let it read the codebase and reason without editing—catch bugs before they exist, not after.
- **Myth:** If I close the terminal, my work is lost. **Reality:** Sessions are saved. Use --continue to reopen the most recent, or --resume to pick a specific named session from the saved list.

Key Takeaway

Claude Code is a tool-using orchestration framework that operates inside your kitchen (your codebase, your terminal, your build tools), not a vending machine narrating recipes from afar. Install it, authenticate, run /init to scaffold a CLAUDE.md, and open your first prompt in plan mode—think before you code.

Quick Quiz

Q1. Which of the following is the strongest mental model for Claude Code?

- A) An autocomplete plugin that lives inside your editor.
- B) A web chat with copy-paste code suggestions.
- C) A CLI that takes natural-language tasks and orchestrates real tools (read, edit, bash, MCP) under your supervision.
- D) A read-only documentation assistant.

Answer: C — Claude Code is a tool-using orchestration framework. The vending machine vs. kitchen analogy: it operates in your kitchen rather than narrating recipes from afar.

Q2. You're starting a long session in a new repo. Which command should you run first?

- A) `/clear`
- B) `/compact`
- C) `/init` — generates a starter `CLAUDE.md`
- D) `/hooks`

Answer: C — `/init` inspects the repo and produces a `CLAUDE.md` scaffold—the starting point for project-specific config.

Q3. You closed your terminal yesterday mid-task. Today you want exactly that conversation back, not a different one. What do you run?

- A) `claude --continue`
- B) `claude --resume` — pick the named session from the list
- C) `claude --new`
- D) `claude /restore`

Answer: B — `--continue` reopens the most recent session, but if you ran multiple yesterday, `--resume` opens a picker so you can choose the right session.

CC1: API Essentials for CLI Power Users

Master models, Messages API, system prompts, temperature, and structured output.

The Big Idea

Claude Code is built on the Messages API, but the CLI hides most of it from you. Understanding the primitives—how models work, how to build conversations, what system prompts do—turns mysteries into straightforward debugging. Every skill, subagent, and hook you write will rely on API concepts. This module exposes the engine bay: model selection, the four required fields of an API request, multi-turn conversation mechanics, system prompts, temperature tuning, streaming, and structured output via tool use.

Analogy

Driving a car works fine until something breaks. That day you need to know what a spark plug and oil pump are. Claude Code is the same: you can ship for months using just the CLI, but the moment a skill won't follow instructions, a subagent picks the wrong model, or a hook returns malformed JSON, you need to understand the Messages API underneath. This module is the engine bay.

How It Works

1. **Pick a model** — Opus 4.7 (deep reasoning), Sonnet 4.6 (default), or Haiku 4.5 (fast, cheap). Pin dated IDs in production.
2. **Build a Messages API request** — Four required fields: model, max_tokens, messages array (alternating user/assistant), optional system prompt.
3. **Maintain conversation history** — To continue a chat, append the previous assistant response and new user message, then re-send the entire array every turn.
4. **Tune with system prompts and temperature** — System prompt = stable role/constraints; temperature 0–0.3 for determinism, 0.7–1.0 for creativity.
5. **Get structured data reliably** — Don't ask for JSON or prefill; use tool use with forced tool_choice for schema-validated responses.

Pseudocode

```
function makeAPICall(model, prompt):
    request = {
        model: model
        max_tokens: 1024
        system: "You are a code reviewer"
        messages: [{role: "user", content: prompt}]
    }
    response = call API with request
    return response.content[0].text

function multiTurnConversation():
    history = []
    user_input = "What is Canberra?"
    history.append({role: "user", content: user_input})

    response = call API with history
    assistant_text = response.content[0].text
    history.append({role: "assistant", content: assistant_text})

    follow_up = "And its population?"
    history.append({role: "user", content: follow_up})

    response = call API with entire history
    return response // Claude remembers Canberra context
```

Common Misconceptions

- **Myth:** The Messages API stores conversation history on the server. **Reality:** Claude is stateless. You send the entire prior conversation on every request; the messages array is your chapter folder.
- **Myth:** You can send two consecutive user messages; the API will concatenate them. **Reality:** Roles must alternate user/assistant. Two user messages = 400 error; merge them into one message instead.
- **Myth:** Asking Claude to "return JSON only" is a reliable way to get structured output. **Reality:** Text-based prompting is fragile. Tool use with forced tool_choice is the only schema-validated approach that guarantees shape.

Key Takeaway

Every Claude Code feature—skills, subagents, hooks, CLAUDE.md integration—is a wrapper over the Messages API. Understanding model selection, the four required request fields, conversation history mechanics, temperature tuning, and tool-based structured output gives you the mental model to reason about why Claude behaves the way it does, debug edge cases, and write production systems that don't surprise you.

Quick Quiz

Q1. Your hook classifies stack traces as "transient" vs "bug" 50,000 times a day. Which model?

- A) Opus 4.7 — high-stakes classification
- B) Sonnet 4.6 — safe default
- C) Haiku 4.5 — classification is bounded, volume is high, latency matters
- D) Whichever the CLI defaults to

Answer: C — Haiku is built for bounded classification at high volume. Picking Opus costs 15x more with no accuracy gain.

Q2. You send `messages=[{role:"user", ...}, {role:"user", ...}]`. What happens?

- A) The two user messages concatenate automatically
- B) 400 error — roles must alternate
- C) Claude treats the second as a system message
- D) The first is silently dropped

Answer: B — The API requires strictly alternating user/assistant turns. Merge extra context into a single user message.

Q3. You're calling Claude in a CI script that diffs two JSON files. What temperature?

- A) 0.0 — classification wants reproducibility
- B) 0.7 — room to think creatively
- C) 1.0 — for thoroughness
- D) Doesn't matter; CI is non-interactive

Answer: A — Determinism is the whole point of CI. Temperature 0 gives stable verdicts across runs, avoiding flaky failures.

CC2: Prompt Engineering for Claude Code

Five techniques to make Claude do the right thing reliably: clarity, specificity, XML structure, examples, thinking.

The Big Idea

Every line you write in Claude Code—CLAUDE.md, skill descriptions, subagent prompts, slash commands—is a prompt. Bad prompts waste tokens and erode trust; good prompts steer Claude reliably on the first try. Five core techniques (clarity, specificity, XML structure, examples, thinking triggers) apply across all surfaces. A well-written prompt can produce 50% accuracy gains without changing the model.

Analogy

Tell a contractor "make the kitchen nicer" and you'll get whatever they think nice means. Tell them "replace laminate counters with quartz, leave cabinets, repaint walls Benjamin Moore Cloud White" and you get exactly that. The same with Claude: vague prompts get different answers each time; direct, specific prompts get the same reliable answer, parsed correctly.

How It Works

1. **Be clear & direct:** Use imperatives ("Find...", "Output..."). State the format up front. Eliminate ambiguity.
2. **Be specific:** Specific inputs with context. Locked output format (parseable). Hard constraints (word counts, enums).
3. **Structure with XML tags:** Wrap regions like `<rules>`, `<input>`, `<example>` and refer to them by name in instructions.
4. **Provide examples:** 3 few-shot examples (typical, ambiguous, known-failure case) teach the shape of correct answers better than rules.
5. **Trigger thinking:** For hard tasks, ask Claude to reason step-by-step or use extended thinking (Claude 4.x) with "think hard" phrases.

Pseudocode

```
// Step 1: Start with a vague prompt
"Review this code and tell me what's wrong."

// Step 2: Add clarity and specificity
"Find only security issues: SQLi, XSS, secrets, auth bypass, IDOR.
Output as bullet list.
Each bullet: filename:line - severity (critical/high/medium).
Skip style issues."

// Step 3: Add XML structure
<rules>
- Find: SQLi, XSS, secrets, auth bypass, IDOR
- Ignore: style, formatting
</rules>

<output_format>
VERDICT: clean | issues
FINDINGS:
- file:line - severity - description
</output_format>

// Step 4: Add a few-shot example
<example>
<input>
SELECT * FROM users WHERE id = $userId; // $userId from URL
</input>
<output>
VERDICT: issues
FINDINGS:
- auth.sql:1 - critical - SQLi: $userId passed unsanitized into query
</output>
</example>

// Result: Claude follows the exact shape, outputs are parseable and
consistent.
```

Common Misconceptions

- **Myth:** "Please" and "thank you" matter. **Reality:** Politeness adds tokens, not accuracy. Claude doesn't need butter-up; direct imperatives are clearer and shorter.
- **Myth:** A better model fixes vague prompts. **Reality:** Clarity and specificity produce 50%+ accuracy gains—often cheaper and faster than upgrading.
- **Myth:** All examples should go in CLAUDE.md. **Reality:** Examples are token-heavy. Put them in skill bodies (loaded on invocation) not CLAUDE.md (loaded every turn).

Key Takeaway

Prompt engineering is not philosophy—it's a set of five learnable techniques: be clear and direct, be specific (inputs, format, constraints), use XML tags, provide 3 examples for classification tasks, and trigger thinking for hard problems. Apply them to every surface: CLAUDE.md (tight rules), skill bodies (structured + examples), subagent prompts (role + format + thinking), and slash commands (locked output). Bad prompts look like Claude is ignoring your rules; usually the rule was too vague to parse.

Quick Quiz

Q1. Your CLAUDE.md says "review code carefully." Reviews come back inconsistent. Best fix?

- A) Switch to Opus.
- B) Replace "carefully" with specific categories: "Look for SQLi, XSS, secrets. Output bullet list."
- C) Add "please be thorough" three times.
- D) Lower temperature.

Answer: B — "Carefully" is unmeasurable. Specific categories + locked output format make Claude consistent. Vagueness is the bottleneck, not the model.

Q2. You want a skill body to apply rules to input. Best structure?

- A) Plain prose: "Apply the following rules to the data..."
- B) XML tags: `<rules>...</rules>` and `<input>...</input>` with instruction "Apply `<rules>` to `<input>`."
- C) JSON: `{rules: [...], input: ...}`
- D) Markdown headers: `# Rules / # Input`

Answer: B — Claude is specifically trained to respect XML tag structure. You can refer to sections by name in instructions, improving instruction-following measurably.

Q3. Your subagent does classification with 0 examples and a vague description. Best first improvement?

- A) Add 20 examples.
- B) Add 3 examples covering typical, ambiguous, and a known-failure case.
- C) Add a "please be accurate" sentence.
- D) Switch model from Haiku to Opus.

Answer: B — Three is the sweet spot for few-shot learning. More than 5 hits diminishing returns and inflates token cost. Pick examples covering edge cases, not volume.

CC3: CLAUDE.md & Memory

How Claude Code remembers things across sessions, repos, and machines via configuration cascades and memory shortcuts.

The Big Idea

CLAUDE.md is a markdown file that holds all the conventions, rules, and domain knowledge for a project. Claude Code reads up to five different CLAUDE.md files (user-level, project-level, directory-level, and org-managed) and merges them in priority order. The cascade solves the problem: "why does Claude follow rule X on my machine but not my teammate's?" Answer: that rule lives in your personal CLAUDE.md, not the project file. Learn where each layer lives, how to write a lean 25-line minimum-viable file, and how to use the # and @ shortcuts to manage memory without leaving your session.

Analogy

Think of how a chef inherits cooking style: the world has general culinary norms (don't cross-contaminate), the restaurant has a house style (we plate French), the kitchen station has its own rules (grill preheats to 500°F), and the chef has personal preferences (always finish with butter). Each layer adds detail; specific layers override general ones. The pain without layering: every chef gets one 100-page manual where specifics about tonight's tasting menu drown in general kitchen safety rules. CLAUDE.md tiers give you those layers for code projects.

How It Works

1. **Five tiers load at session start:** Managed (org policy) → Local (your machine, .claude/CLAUDE.local.md) → Project (./CLAUDE.md, checked into git) → Directory (src/payments/CLAUDE.md, any subdirectory) → User (~/.claude/CLAUDE.md, your machine only). Tier 1 wins on conflict; tiers below add and refine.
2. **Write the minimum-viable project CLAUDE.md:** five sections (What This Project Is, How to Run Things, Conventions, Don't Do These, On Commits), roughly 25 lines. Keep it under 200 lines total; long playbooks go in skills, which load only when invoked.
3. **Use path-specific rules for monorepos:** either via frontmatter globs in a single CLAUDE.md, or per-directory CLAUDE.md files. When Claude edits code in packages/api/src/billing/, it gets the union of all CLAUDE.md files in that path tree.
4. **Edit memory in-session without leaving:** /memory shows all loaded files and lets you pick one to edit. # at message-start saves a memory line on the fly. @ paths inside CLAUDE.md import other markdown files (up to 5 hops deep).

5. **Avoid the four anti-patterns:** don't restate your README, don't embed long tutorials (move to skills), don't add personality instructions ("you are a helpful assistant" is noise), and never commit secrets like API keys or database passwords.
-

Pseudocode

At session start, Claude Code performs the cascade:

```
IF managed-policy exists:
  LOAD managed-policy
IF .claude/CLAUDE.local.md exists:
  MERGE .claude/CLAUDE.local.md (adds/overrides managed)
IF ./CLAUDE.md exists in current repo:
  MERGE ./CLAUDE.md (adds/overrides local)
FOR each directory from current file up to repo root:
  IF CLAUDE.md exists in that directory:
    MERGE that CLAUDE.md (adds/overrides project level)
IF ~/.claude/CLAUDE.md exists:
  MERGE ~/.claude/CLAUDE.md (adds to everything else)

FINAL merged instructions are now loaded in this session.
When editing a file, nearest CLAUDE.md wins on any conflict.
```

Common Misconceptions

- **Myth:** CLAUDE.md should contain everything from the README, plus tips, plus tutorials. **Reality:** CLAUDE.md should be lean and fast, loaded every turn. Long playbooks and tutorials belong in skills (CC5), which load only when invoked. Keep CLAUDE.md under 200 lines.
- **Myth:** When two CLAUDE.md files conflict, the newer/recent one wins. **Reality:** Tier number wins. Project (tier 3) beats user (tier 5) on any conflict, regardless of recency. Know the cascade order: Managed > Local > Project > Directory > User.
- **Myth:** User-level CLAUDE.md (~/.claude/CLAUDE.md) applies to teammates too. **Reality:** It's your machine only. To share conventions with teammates, move rules to ./CLAUDE.md (project level) and check it into git.

Key Takeaway

CLAUDE.md is not a prompt or a tutorial file — it's a standing instruction that Claude Code loads every session, telling Claude what conventions and rules apply to this project. The five-tier cascade solves "why do different people see different behavior" by letting org policies flow down through team rules, project rules, sensitive-area rules, and personal preferences. Write a lean 25-line project file, use directory-level files for monorepos, and use the # and @ shortcuts to manage memory without leaving your session.

Quick Quiz

Q1. Two CLAUDE.md files conflict: ~/.claude/CLAUDE.md says "always show diffs" and ./CLAUDE.md says "skip diffs for trivial changes." Which wins?

- A) User-level file (always show diffs)
- B) Project file (skip for trivial) — project tier 3 beats user tier 5
- C) They concatenate; both apply
- D) Last-loaded wins

Answer: B — The cascade prioritizes by tier number. Project (tier 3) beats user (tier 5). Lower tier number = higher priority.

Q2. You're in a monorepo. packages/billing/CLAUDE.md says "all changes need tests." ./CLAUDE.md says nothing. You edit a file in packages/billing/. What rules apply?

- A) Only root CLAUDE.md
- B) Only billing CLAUDE.md
- C) Both — directory file adds to root rules
- D) Neither — CLAUDE.md only loads at repo root

Answer: C — Per-directory CLAUDE.md files are additive. They merge with root. The deeper file can tighten rules without bloating the root file.

Q3. Your CLAUDE.md is 800 lines and Claude responses are getting slow. What's the right fix?

- A) Trim everything not strictly required
- B) Use /compact more often
- C) Move long playbooks into skills; keep only always-on rules in CLAUDE.md
- D) Switch to a faster model

Answer: C — CLAUDE.md content loads every turn. Skills load only when invoked. For long playbooks, skills are far cheaper. Trimming helps too, but skills are the structural fix.

CC4: Permissions & Sandbox

Two independent safety layers: ask for permission, enforce boundaries with OS-level jail.

The Big Idea

Claude Code has two independent safety mechanisms stacked on top of each other. Permission modes are advisory—they tell Claude to ask you before executing certain tools or commands. The sandbox is OS-enforced—it's an operating-system-level jail that prevents any process from reading or writing outside the paths you allow, regardless of what Claude or bash tries to do. Together, they create defense in depth: permission modes handle day-to-day UX (you control when you're prompted), and the sandbox catches mistakes the permission layer misses.

Analogy

A research lab uses two layers: posted procedures ("wear gloves, work in the fume hood") and physical engineering (the fume hood actually exists, the gas valve is locked). The procedure is advisory; the engineering is mandatory. If you only have the procedure, a careless person ignores it. If you only have the engineering, every routine task feels locked down. You need both, with engineering catching what procedure misses.

How It Works

1. You define **permission modes** (plan, default, acceptEdits, bypassPermissions) per session, controlling whether Claude prompts you before running bash, editing files, or fetching URLs.
2. You define **permission rules** (allow, ask, deny) in settings.json to pre-approve specific commands (`Bash(npm test:*)`), force prompts for others (`Bash(git push:*)`), or block them entirely (`Bash(rm:*)`).
3. You optionally enable the **sandbox**, an OS-level jail that whitelists filesystem paths Claude can write to and network domains it can reach. Even if Claude misbehaves, the OS enforces the boundary.
4. Rules are layered: user-level settings apply globally, project-level settings override them for one repo, and sandbox is independent and can work alone or in combination with either.
5. Precedence is deny > ask > allow: if a command matches a deny rule, it's blocked; if it matches ask, you're prompted; if only allow, it runs without prompting.

Pseudocode

```
// Settings.json structure
{
  "permissions": {
    "defaultMode": "default",      // plan | default | acceptEdits |
    bypassPermissions
    "allow": [
      "Bash(npm test:*)",          // Auto-run test commands
      "Bash(git diff:*)",          // Auto-run git diffs
      "Read(./**)"                 // Auto-read files in project
    ],
    "ask": [
      "Bash(git push:*)",          // Prompt before pushing
      "Edit(package.json)"         // Prompt before editing package.json
    ],
    "deny": [
      "Bash(rm:*)",                // Block rm commands entirely
      "Read(~/.ssh/**)",           // Block reading SSH keys
      "Read(./env)"                // Block reading .env file
    ]
  },
  "sandbox": {
    "enabled": true,
    "filesystem": {
      "allowWrite": ["./", "~/.cache/claude"], // Only write here
      "denyRead": ["./.env", "~/.ssh", "~/.aws"] // Never read these
    },
    "network": {
      "allowedDomains": ["api.github.com", "registry.npmjs.org"],
      "deniedDomains": ["*"] // Default-deny, then whitelist
    }
  }
}
```

Common Misconceptions

- **Myth:** I should use `bypassPermissions` on my dev machine for speed. **Reality:** Never. One hallucination or typo runs `rm -rf` with no prompt. Use `bypassPermissions` only in disposable containers (Docker, GitHub Actions, throw-away VMs) where the blast radius is bounded by the container itself.
- **Myth:** Permission modes and sandbox do the same thing. **Reality:** Permissions are advisory (Claude cooperates); sandbox is OS-enforced (the operating system blocks the action). Permissions handle UX; sandbox handles security. Use both for defense in depth.
- **Myth:** If I allow a broad tool like `Bash`, deny rules won't help. **Reality:** Deny beats allow. `"allow": ["Bash"]` paired with `"deny": ["Bash(rm:*)"]` blocks `rm` even though `Bash` is allowed. Pair broad allows with narrow denies.

Key Takeaway

Permission modes and sandbox are independent safety layers. Use permission modes to tune your day-to-day workflow (what prompts you), and use the sandbox to enforce hard boundaries that the OS itself enforces. Never use `bypassPermissions` on your dev machine. Layer both for defense in depth: the permission layer stops careless mistakes; the sandbox catches what permission layer misses.

Quick Quiz

Q1. You're about to refactor authentication across 12 files. Which mode?

- A) `acceptEdits` — speed matters
- B) `plan` — read-only, get a written plan first
- C) `bypassPermissions` — you're experienced
- D) `default` — just answer prompts as they come

Answer: B — Plan mode lets you read and think but not write or execute. You get a full plan first, review it for architectural issues, then drop into default mode to execute. Fastest way to catch big mistakes before code is written.

Q2. Your settings have `"allow": ["Bash"]` and `"deny": ["Bash (rm:*)"]`. Claude tries `rm -rf node_modules`. What happens?

- A) Auto-runs — matches allow first
- B) Blocked — deny beats allow
- C) Prompts — rules conflict
- D) Error returned to Claude without telling you

Answer: B — Precedence is deny > ask > allow. Pair broad allows with narrow denies for destructive patterns. This is how you safely allow Bash for tests while blocking rm.

Q3. You enable the sandbox with `filesystem.allowWrite: ["/./"]` . A script tries to write to `/tmp/cache` . What happens?

- A) Writes succeed — /tmp is special-cased
- B) Write fails (EACCES) — /tmp is outside allowlist
- C) Claude prompts you to allow
- D) It's redirected silently into the project

Answer: B — Sandbox is OS-enforced. Only paths in `allowWrite` are writable; everything else returns an OS error (EACCES). If you need `/tmp/cache`, add it to `allowWrite` explicitly. This is what makes sandbox different from permission modes: it's not advisory, it's mandatory.

CC5: Skills & Slash Commands

Build custom commands as unified skills; use SKILL.md, shell injection, and fork execution.

The Big Idea

Custom commands and skills merged into one system. A skill is a directory with a SKILL.md file containing YAML frontmatter and a prompt body. Old .claude/commands/*.md files still work as degenerate skills. Skills live in 4 scopes (Enterprise, Personal, Project, Plugin) with inheritance rules. You can inject live shell data, run skills in isolated fork contexts, and access 6 substitution variables. New work gets the full skill upgrade; old work keeps running unchanged.

Analogy

Your kitchen used to have two recipe systems: index cards (one-pagers) and binders (multi-page playbooks with prep notes and photos). Both produced dishes; the split forced you to choose format per task. Then cards evolved: stick a card in a folder, optionally add supporting files, same place and name, just more capacity. That's the merge—old cards still work, new ones get the upgrade for free.

How It Works

1. Create .claude/skills/my-skill/ directory with SKILL.md inside.
2. Write YAML frontmatter (description, allowed-tools, context, etc.) and a Markdown prompt body.
3. Trigger via /my-skill (slash command) or let Claude auto-invoke if description matches context.
4. Use substitution variables (\$ARGUMENTS, \$N, \$name, \${CLAUDE_SESSION_ID}, \${CLAUDE_EFFORT}, \${CLAUDE_SKILL_DIR}) in the prompt.
5. Optionally inject shell output with !`cmd` syntax, isolate via context: fork, or pre-approve tools with allowed-tools.

Pseudocode

```
IF slash_command_typed THEN
  Lookup skill by name in hierarchy: Enterprise > Personal > Project
  IF skill found THEN
    Render SKILL.md: substitute arguments, execute !`cmd` blocks,
    interpolate variables
    Send rendered prompt to Claude
    IF context: fork THEN
      Run in isolated subagent; return summary only
    ELSE
      Run inline; tool calls visible in main context
    END
  ELSE
    Try old .claude/commands/name.md (backward compat)
  END
ELSE IF Claude_decides_to_auto_invoke THEN
  Match description against context; fire if confident
END
```

Common Misconceptions

- **Myth:** allowed-tools blocks other tools. **Reality:** It only pre-approves those listed; Claude can still call other tools (they trigger permission prompts). To actually deny, use permissions.deny.
- **Myth:** !`cmd` is executed by Claude during tool calls. **Reality:** It runs at skill render time, before Claude sees the prompt; output is inlined as static text.
- **Myth:** context: fork works without a task. **Reality:** Fork isolation only works if the skill has a clear task; pure reference content should run inline so Claude uses it alongside your conversation.

Key Takeaway

Skills unify commands and enable team playbooks. Drop a SKILL.md anywhere in the 4-scope hierarchy and Claude picks it up live. Use fork execution for isolated research, shell injection for live data, and substitution variables to parameterize reusable prompts. Skills convert tribal knowledge into checked-in artifacts your whole team runs consistently.

Quick Quiz

Q1. You have `.claude/commands/deploy.md` and your teammate adds `.claude/skills/deploy/SKILL.md`. What runs when you type `/deploy`?

- A) Error—name conflict.
- B) The command (older) wins.
- C) The skill wins; skills take precedence over same-named commands.
- D) The most recently modified file wins.

Answer: C — Skill takes precedence. Old commands keep working, but if both exist at the same scope, skill wins.

Q2. Your skill needs the GitHub PR diff inlined into the prompt without a tool round-trip. What is the right syntax?

- A) `{{ gh pr diff }}`
- B) `!`gh pr diff`` — inline shell injection
- C) `$(gh pr diff)`
- D) Not possible; tools must round-trip.

Answer: B — `!`cmd`` runs at render time and inlines stdout. For multi-line, use a fenced ```!`` block.

Q3. A skill must do thorough codebase search but you do not want all file contents flooding main context. What do you set?

- A) `disable-model-invocation: true`
- B) `allowed-tools: Read`
- C) `context: fork with agent: Explore`
- D) Move it to `~/.claude/skills/`

Answer: C — Fork isolation = `context: fork`. The Explore subagent type is read-only and tuned for codebase search; only the summary returns to your main session.

CC6: Subagents

Specialist agents in isolated contexts—delegate verbose work, keep your conversation clean and focused.

The Big Idea

Subagents are specialist workers that run in their own context window with their own tools and optional memory. When you need to run a task that produces massive output but returns just a summary—scanning the codebase, auditing for patterns, multi-step research—hand it to a subagent. The subagent's reads, greps, and tool calls stay out of your main conversation. You get clean summaries. It's the architectural equivalent of delegation: stay focused on the high-level conversation while specialists handle the verbose details.

Analogy

Imagine asking a junior architect to verify that your database schema matches the OpenAPI spec. You could read all 47 migration files and 12 endpoint definitions yourself, spending three hours. Better: hand it to the junior, let them work in their own corner, and have them come back with "consistent, except orders.status drifts from OrderStatus." That verdict is gold. Their process (the 47 files they read) doesn't show up in your office.

Subagents are that junior. Each runs in a fresh context, gets a tight prompt and tool allowlist, and returns just the summary.

How It Works

1. **Author the subagent** as a Markdown file with YAML frontmatter at `.claude/agents/<name>.md`. Define its name, description, allowed tools, model, and optional memory scope.
2. **Claude detects** when your task matches the subagent's description and spawns it in a fresh context window with just that focused prompt.
3. **Subagent runs** in isolation—reads files, greps patterns, edits in a worktree—without those tool calls appearing in your main transcript.
4. **Result returns** to you as a clean summary. Your conversation stays high-level; verbose work stays hidden.
5. **Memory persists** (optional) across sessions so the subagent learns codebase patterns and gets smarter over time.

Pseudocode

```
When task matches subagent description:
  create new context window
  inject subagent frontmatter as system prompt
  inject MEMORY.md (first 200 lines) if memory scope set
  enable only tools listed in frontmatter
  submit user task to subagent

While subagent works:
  tool calls don't appear in parent transcript
  if tool_call not in allowed_tools: refuse

When subagent finishes:
  return summary to parent
  if memory: project set: append new learnings to MEMORY.md
  close subagent context
  parent conversation resumes clean
```

Common Misconceptions

- **Myth:** Subagents are just skills with memory. **Reality:** Subagents run in isolated context windows; their tool calls are invisible. Skills run inline in your conversation. Pick subagent for verbose work you don't want to see; pick skill for reusable logic you might invoke multiple times in one session.
- **Myth:** All five built-in subagents (Explore, Plan, general-purpose, statusline-setup, Claude Code Guide) fire automatically. **Reality:** Only those whose `description` matches your current task. You can disable any with `"deny": ["Agent (Explore)"]` in permissions.
- **Myth:** Plugin subagents can include hooks and MCP server definitions. **Reality:** Plugin subagents silently drop `hooks`, `mcpServers`, and `permissionMode` for security. Copy the agent to `.claude/agents/` if you need them.

Key Takeaway

Subagents isolate verbose work. A subagent runs in its own context, respects a narrow tool allowlist, and returns just the summary. Use them when a task's internal process would drown your conversation—scanning, auditing, multi-step research, code review. Author one as a Markdown file with 16 frontmatter fields; invoke via natural language, @-mention, or `--agent` flag. Optional persistent memory makes the subagent learn across sessions.

Quick Quiz

Q1. You want a subagent that learns about your codebase across sessions, with knowledge shareable to your team via git. Which `memory:` value?

- A) `user`
- B) `project`
- C) `local`
- D) `global`

Answer: B — `project` stores memory at `.claude/agent-memory/<name>/`, which goes into git. `user` is personal-only; `local` is project but gitignored.

Q2. The parent session runs in `bypassPermissions` mode. The custom subagent's frontmatter says `permissionMode: default`. What actually happens?

- A) Subagent uses `default` — frontmatter wins.
- B) Subagent inherits `bypassPermissions` from parent — cannot be overridden.
- C) Error — modes conflict.
- D) The stricter of the two wins.

Answer: B — If the parent uses `bypassPermissions`, `acceptEdits`, or `auto` mode, that takes precedence and the subagent's `permissionMode` frontmatter is ignored.

Q3. Subagent vs Skill vs Main—where does "verbose research that produces a 5-line summary" go?

- A) Main conversation — you need to see the work.
- B) Skill running inline — reusable.
- C) Subagent (or skill with `context: fork`) — verbose work isolated, summary returns.
- D) Plan mode in main — describe the work first.

Answer: C — The classic subagent use case: heavy verbose work in isolation, just the summary returned. Either dedicated subagent or a fork-style skill both achieve isolation.

CC7: Hooks

Programmatic control over tool calls. Auto-format, block, route policy, inject context.

The Big Idea

Hooks are thin layers that intercept Claude's tool calls before and after they run. Instead of Claude executing a Bash command directly, your hook runs first, checks policy, logs the call, formats output, or blocks it entirely. Think of them as middleware for agent behavior: same Claude, same tools, but with guardrails and automation you control. The 9 hook events (SessionStart, SessionEnd, UserPromptSubmit, PreToolUse, PostToolUse, Stop, SubagentStop, PreCompact, Notification) let you enforce policy across the entire lifecycle. Configuration lives in `.claude/settings.json`.

Analogy

Every hotel room has card readers instead of traditional locks. Why? Because the front desk revokes access without fishing for keys, logs every entry centrally, and updates rights when guests extend their stay—all without changing the door itself. Hooks in Claude Code are exactly this: a thin layer around tool calls that runs *your* code first. Claude doesn't know it's there; you control whether the call proceeds, what gets logged, or whether the output is reformatted.

How It Works

1. **Event fires:** Claude is about to use Bash (`PreToolUse`) or just finished an Edit (`PostToolUse`).
2. **Hook runs:** Your command, HTTP endpoint, or LLM prompt executes with the tool call JSON on stdin.
3. **Decision returned:** Hook exits 0 (allow), 2 (block), or returns JSON with `permissionDecision: "allow" | "deny" | "ask" | "defer"`.
4. **Claude acts:** If blocked, Claude sees the rejection and tries something else. If allowed, the tool runs. If deferred, the session parks until a slow decision lands.
5. **Output handled:** After tool completion, PostToolUse hooks can lint, format, or inject system reminders back to Claude.

Pseudocode

```
Configuration in .claude/settings.json {
  hooks: {
    PreToolUse: [           // event: before tool runs
      {
        matcher: "Bash",    // only Bash tool calls
        hooks: [
          {
            type: "command", // run shell script
            command: "check-dangerous-flags.sh"
          }
        ]
      }
    ],
    PostToolUse: [          // event: after tool runs
      {
        matcher: "Edit|Write",
        hooks: [
          {
            type: "http",    // POST to policy server
            url: "https://policy.internal/lint",
            async: true      // fire and forget
          }
        ]
      }
    ]
  }
}
```

Hook script (check-dangerous-flags.sh) reads stdin:

```
tool_name = "Bash"
tool_input.command = "rm -rf /"
hook_event_name = "PreToolUse"

If tool_input.command contains "rm -rf":
  exit 2 // block signal
  print to stderr: "Dangerous"
Else:
  exit 0 // allow
  optionally return JSON with changes to tool_input
```

Common Misconceptions

- **Myth:** "Hooks run *instead of* the tool." **Reality:** `PreToolUse` runs *before* the tool; `PostToolUse` runs *after*. Only `PreToolUse` can block. `PostToolUse`'s exit 2 is feedback to Claude, not a block.
- **Myth:** "I need a different hook script for each developer." **Reality:** Use `type: "http"` to centralize policy at one endpoint. One server, one audit log, one source of truth. Update policy once; it applies to everyone.

- **Myth:** "Hooks are for permissions only." **Reality:** Hooks automate formatting (prettier post-Edit), run tests (post-Write), inject context (prompt classification), and manage approval workflows (`permissionDecision: "defer"` for Slack approvals).

Key Takeaway

Hooks let you automate and govern Claude's tool calls without touching the tools themselves. Start with a simple PostToolUse hook to auto-format files, then graduate to HTTP-based centralized policy for orgs. Five handler types (command, http, mcp_tool, prompt, agent) give you options from cheap-and-fast to deep-and-semantic. Remember: exit 2 to block on PreToolUse; exit 0 or JSON to communicate on PostToolUse.

Quick Quiz

Q1. You want to auto-format every file Claude edits. Which hook event?

- A) PreToolUse with matcher: Edit|Write
- B) PostToolUse with matcher: Edit|Write
- C) SessionEnd
- D) UserPromptSubmit

Answer: B — PostToolUse fires *after* the edit completes, perfect for formatting. PreToolUse fires before the file exists in its new state.

Q2. You want to block any Bash call containing rm -rf. What exit code?

- A) exit 1 — standard error
- B) exit 2 — with stderr explaining why
- C) exit 0 with empty output
- D) exit 127 — command not found

Answer: B — Exit 2 is the special blocking signal. On blockable events (like PreToolUse), Claude Code respects it and feeds stderr back to the model so it knows why and can retry.

**Q3. Your team needs one audit log, one policy server, shared enforcement.
Which handler type?**

- A) command — shell script per dev
- B) http — POST to a central policy endpoint
- C) prompt — ask Haiku each time
- D) agent — subagent for each call

Answer: B — HTTP hooks centralize policy in one server with one audit trail.

Command means N copies on N machines. Don't forget `allowedHttpHookUrls`
and `httpHookAllowedEnvVars` in settings.

CC8: Tool Use Deep Dive

Master the four-step tool-use loop, multi-tool orchestration, and fine-grained control.

The Big Idea

Tool use transforms Claude from a text-only model into an agent. The mechanism is a four-step loop: your app sends Claude a user message plus tool definitions; Claude responds with a `tool_use` block (decide to call); your app runs the tool and returns results; Claude integrates and answers. This loop repeats until done. Every Claude Code action — reading files, editing, running Bash, searching — is tool use under the hood. Understanding this loop is understanding how agents work.

Analogy

Imagine a doctor diagnosing without ordering tests — just guessing from symptoms. Now give that doctor a lab next door: order a CBC, get results, decide. Slower per question, dramatically more accurate. Tool use is the lab. Claude alone is the doctor without one; with tools, Claude can look things up, run code, edit files, query databases. Each tool call is an order; the result comes back and Claude integrates it.

How It Works

1. App defines tools (name, description, input schema) and sends user message + tool list to Claude.
2. Claude responds with `tool_use` block containing tool name and input. Stop reason becomes "tool_use".
3. App executes the tool function and builds a `tool_result` block with output (or error) and `tool_use_id`.
4. App re-sends full history (assistant's prior content + new `tool_result` in a user-role message). Claude integrates and returns text.

Pseudocode

```
define tools with name, description, input schema
send user message + tools to Claude

loop until stop_reason != "tool_use":
    receive assistant response

    for each tool_use block in response:
        execute tool function with input
        create tool_result block linking to tool_use_id

    append assistant's full prior content to history
    append user message containing all tool_result blocks
    re-send request with updated history

return final text from Claude
```

Common Misconceptions

- **Myth:** You can call the tool yourself and skip sending the result back. **Reality:** Claude must see the full prior tool_use block and the matching tool_result in history or the loop breaks and it can't match IDs.
- **Myth:** Tool results should go in an assistant-role message. **Reality:** tool_result always goes inside a user-role message, as one item in a content array that can have multiple results.
- **Myth:** If a tool fails, throw an exception and let your app handle it. **Reality:** Return a tool_result with is_error: true so Claude stays inside the loop and can retry or adapt gracefully.

Key Takeaway

The tool-use loop is the heart of agentic AI. Request → tool_use decision → run tool → send result back → Claude answers. Understanding this loop explains everything Claude Code does: its Bash, Edit, Read, Grep tools all follow this protocol. MCP servers and skills add more entries to the same tools array.

Quick Quiz

Q1. Your tool call returns a dictionary with nested fields. How should you send it back to Claude?

- A) Send it as a Python object
- B) JSON stringify it so Claude can reason about fields
- C) Just convert to string
- D) Wrap it in another dict

Answer: B — JSON stringify structured returns. Claude reads tool results as strings, but JSON format lets it parse and reason about nested fields.

Q2. Claude emitted two `tool_use` blocks in one assistant message. How do you respond?

- A) Send two separate follow-up requests
- B) Run both tools and send one user message with both `tool_result` blocks
- C) Pick one and ignore the other
- D) Combine the outputs into a single `tool_result`

Answer: B — Parallel tool use. Run both and reply with a single user-role message containing both `tool_result` blocks, each with its matching `tool_use_id`. Claude consumes them as one turn.

Q3. You want to force Claude to always call your `record_signoff` tool. Best approach?

- A) Set `tool_choice`: `{"type": "auto"}`
- B) Set `tool_choice`: `{"type": "any"}`
- C) Set `tool_choice`: `{"type": "tool", "name": "record_signoff"}`
- D) Set `tool_choice`: `{"type": "none"}`

Answer: C — Forcing a named tool requires `{"type": "tool", "name": ...}`. Option A is default (Claude decides). Option B means "some tool" but not which one. Option D disables tools.

CC9: MCP — Model Context Protocol

Wire GitHub, Postgres, Slack, and custom tools into Claude Code as first-class capabilities using the open MCP protocol.

The Big Idea

MCP (Model Context Protocol) is an open standard from Anthropic that lets AI assistants access external tools and data. Before MCP, integrating a new data source meant building separate plugins for each AI tool. Now, build one MCP server and every AI tool that speaks the protocol can use it—Claude Code, Cursor, ChatGPT. MCP servers expose three primitives: tools (functions Claude can call), resources (read-only data), and prompts (template workflows). Three transports carry the protocol: stdio (subprocess, the default), HTTP, and Server-Sent Events. The production ecosystem includes GitHub, Postgres, Slack, Jira, Playwright, and dozens of community servers.

Analogy

MCP is USB-C for AI tools. Before USB-C, every phone had its own charger. Now one cable works everywhere. Before MCP, every AI needed its own GitHub plugin, Slack plugin, database plugin. Now build one MCP server and the whole ecosystem benefits.

How It Works

1. **Define the server:** Write Python code using FastMCP that exposes tools (functions with descriptions), resources (URI-addressed data), and prompts (parameterized templates).
2. **Register with Claude Code:** Drop a `.mcp.json` file in your project root with the server's command and transport, or use ``claude mcp add`` to auto-generate it. You can scope to project (shared in git), user (personal across projects), or managed (organization-wide).
3. **Trust on first run:** When Claude Code sees a new `.mcp.json`, it prompts: "Project wants MCP servers. Allow?" Pick servers individually; "allow all" is rarely right for unfamiliar repos.
4. **Call from Claude:** The protocol automatically surfaces tools and resources to the model. Ask in plain English and Claude calls the right tool; you see the JSON-RPC traffic in the tool ribbon.
5. **Apply security:** Use narrow tokens (e.g., GitHub PAT with minimal scopes), restrict tool surface via permission rules (default-deny on ``mcp__server__*``, allow only safe operations), audit via `PreToolUse` hooks.

Pseudocode

```
// Minimal MCP server in Python (FastMCP)
import FastMCP

create server "ucc_lookup"

define tool "lookup_filing(po_number)"
    returns {po_number, debtor, status, filed_at, lien_count}
    // real implementation queries Postgres

define tool "search_debtor(name, limit=10)"
    returns list of {po_number, debtor, ...}

define resource "filing://{po_number}"
    uri pattern that returns filing as JSON

define prompt "filing_summary(po_number)"
    template: "Summarize filing {po_number} in 3 sentences"

run server over stdio (default)

// Register in project .mcp.json
{
  "mcpServers": {
    "ucc": {
      "type": "stdio",
      "command": "python server.py",
      "env": { "DB_URL": "${POSTGRES_URL}" }
    }
  }
}

// Use from Claude Code: no shell needed
// "Top 5 states by UCC filings"
// Claude calls: ucc.query(sql_generated)
// Permission rules restrict to read-only
```

Common Misconceptions

- **Myth:** "MCP is for complex integrations." **Reality:** A useful MCP server is 30 lines of Python. FastMCP hides JSON-RPC; you just write functions with docstrings.
- **Myth:** "I should use a Skill instead of MCP." **Reality:** Skills suit prompt-shaped workflows (shell wrappers, one-off curl calls). Use MCP for stateful connections (database sessions, browser instances) and cross-tool reuse. If you're wrapping `gh` or `git`, a skill is simpler; if you need Postgres open the whole session, use MCP.
- **Myth:** "enableAllProjectMcpServers is fine—trust prompts are annoying." **Reality:** Auto-approving every project MCP server bypasses the trust prompt. If a teammate commits new .mcp.json servers later, they auto-run without you reviewing them. Stick

with per-project approval.

Key Takeaway

MCP is the open standard for connecting AI tools to external systems. Three security pillars: narrow tokens (don't reuse everyday credentials), restrict tool surface (permission rules default-deny), audit via hooks. Build once with FastMCP, deploy across any MCP-aware client. The protocol is JSON-RPC 2.0; know the wire format for debugging.

Quick Quiz

Q1. Your team needs Claude Code to read live PostgreSQL data and post Slack messages. Should you write two Skills or use MCP?

- A) Two skills—one calling `psql`, one calling Slack curl
- B) MCP—both need stateful connections; postgres-mcp + slack-mcp
- C) One skill calling both via shell
- D) Subagent only, no MCP

Answer: B — Both need stateful connections (DB session, OAuth-aware Slack client). MCP is built for that. Skills suit thin shell wrappers, not persistent connections.

Q2. You add a third-party MCP server that exposes 30 tools, but you only need 3 read-only ones. How do you restrict the surface?

- A) Edit the server's source to delete the other 27
- B) Use `permissions.deny: ["mcp\_server\_\*"]` + `allow: ["mcp\_server\_list\_\*", "mcp\_server\_read\_\*"]`
- C) Set `disabledMcpjsonServers: ["server"]`
- D) You can't—MCP is all-or-nothing

Answer: B — Default-deny on the server prefix, then explicit allow for the safe subset. Permission rules apply to MCP tools the same way they do to built-in tools (CC4).

Q3. Your subagent needs Playwright but you don't want Playwright tools in your main session's context window. What's the pattern?

- A) Add it to project ``.mcp.json``
- B) Add it to user `~/claude.json``
- C) Inline in the subagent's frontmatter under ``.mcpServers``—scoped to that subagent only
- D) Spawn it as a child process from the subagent's prompt

Answer: C — Inline server definitions in subagent frontmatter exist only while that subagent runs. Main session never sees tool descriptions, doesn't pay context cost. This is the pattern from CC6.

CC10: Claude Features — Thinking, Caching, Citations, Files

Extended reasoning, cost-cutting caching, document input, citations, and sandboxed code execution

The Big Idea

Claude 4.x ships five production-grade features that make apps faster, cheaper, and more trustworthy. Extended thinking gives you a hidden reasoning trace for hard problems. Prompt caching cuts input costs 90% on stable prefixes. Citations grounds answers in source documents automatically. Image/PDF input lets Claude analyze visual content. Code execution runs sandboxed Python. Each solves a real pain point; knowing when to use each is what separates prototype from production.

Analogy

Extended thinking is like asking a senior engineer a hard question and watching them think aloud for two minutes before giving a confident answer—the reasoning happens before the response, invisible but real. Prompt caching is like a tax accountant keeping your filing on their desk for five minutes instead of re-reading it each time you ask "what's box 14b?"—same answer, fraction of the cost. Citations work like a legal brief with footnotes that actually point to the right page, auto-generated, no parsing needed.

How It Works

1. Enable extended thinking per request with a `budget_tokens` parameter; Claude emits hidden thinking blocks before its visible answer.
2. Mark stable content (system prompt, tools, large documents) with `cache_control` metadata; subsequent identical requests reuse the cache at 10% input cost.
3. Pass images or PDFs in message content blocks; Claude analyzes them as part of normal reasoning.
4. Enable citations on document blocks; Claude automatically annotates its answer with source spans.
5. Call the code-execution tool; Claude writes Python, the sandbox runs it, and returns output to embed in the answer.

Pseudocode

```
// Extended thinking for hard reasoning
response = call_claude(
  messages: [user_query],
  thinking: { enabled: true, budget_tokens: 8000 }
)
// response.content has [thinking_block, text_block]

// Prompt caching: mark the stable prefix
response = call_claude(
  system: [
    { type: "text", text: LONG_SYSTEM_PROMPT, cache_control: "ephemeral" }
  ],
  messages: [current_query]
)
// First call: cache_creation_input_tokens filled
// Second call (within 5 min): cache_read_input_tokens filled, 90% savings

// Image and PDF input
response = call_claude(
  messages: [{
    content: [
      { type: "image", source: { type: "base64", data: image_b64 } },
      { type: "text", text: "What's in this image?" }
    ]
  }]
)

// Citations on documents
response = call_claude(
  messages: [{
    content: [
      { type: "document", source: { type: "base64", data: pdf_b64 },
        citations: true },
      { type: "text", text: "Summarize this policy." }
    ]
  }]
)
// response.content[0].citations = array of {cited_text, start_char, end_char}

// Code execution
response = call_claude(
  tools: [{ type: "code_execution", name: "code_execution" }],
  messages: [{ content: "Generate a histogram of
[1,1,2,3,3,3,4,5,5,5,5,7]" }]
)
// Claude writes Python, sandbox runs it, returns figure or file
```

Common Misconceptions

- **Myth:** "Extended thinking always makes answers better." **Reality:** Only on hard reasoning (architecture, refactoring, ambiguous specs). On lookups or classification, thinking wastes tokens. Save it for tasks worth an Opus call.
- **Myth:** "Caching always saves money." **Reality:** Cache writes cost 25% more than normal input. You only profit if you get cache hits later. One-shot calls lose money with caching enabled.
- **Myth:** "My cached prefix is broken—Anthropic's API is buggy." **Reality:** Caches match byte-for-byte. One space, one newline, one timestamp difference and you bust the cache. Keep the prefix static between calls.

Key Takeaway

These five features are how you ship Claude at scale: thinking for hard reasoning, caching for cost, images/PDFs for rich input, citations for trustworthiness, and code-execution for automation. Each one has a gotcha (budget limits, cache invalidation, file size caps, network isolation), but once you know them, they're the difference between a chatbot and a production system.

Quick Quiz

Q1. You're building a file-save hook that needs a fast allow/block decision. Should you enable extended thinking?

- A) Yes, better decisions are always worth it.
- B) No, thinking adds latency and cost; quick verdicts don't need it.
- C) Only on Haiku.
- D) Always at max budget.

Answer: B — Hooks need speed. Extended thinking adds seconds of latency and real token cost. Save thinking for hard reasoning, not snap verdicts.

Q2. Your one-shot CLI tool calls Claude once per run, no follow-up. Should you cache the system prompt?

- A) Yes, caching is always cheaper.
- B) No, you pay the write premium with no second call to amortize it.
- C) Yes, with 1-hour TTL.
- D) Yes, but only on Opus.

Answer: B — Cache writes cost more than normal input. They pay off only if you get hits later. One-shots get no benefit, so caching loses you money.

Q3. You're caching a system prompt but sometimes get cache hits and sometimes don't. Most likely cause?

- A) API outage.
- B) You're concatenating a timestamp or request ID into the prefix.
- C) You changed the model.
- D) Caching is non-deterministic.

Answer: B — Caches match prefix-exactly. Any varying token in the cached region (timestamps, IDs, random data) busts the cache. Move all dynamic content after the cache breakpoint.

CC11: Evaluating Skills, Subagents & Prompts

Treat your skills and prompts like code: test them, measure regressions, gate merges.

The Big Idea

Prompt engineering lives on a spectrum from "vibes" to "science." Without automated evals, changes that feel smarter often aren't—they just overfit. An eval is a test suite for prompts: run your prompt against 20–100 real cases with known-good outputs, measure pass rate, and wire the score into CI to block regressions. Four stages: (1) build a dataset, (2) run the prompt on each case, (3) grade the output (code-based for classification, model-based for subjective), (4) aggregate and gate. The output is a number—"84/100 correct"—that makes prompt changes auditable.

Analogy

Refactoring 10K lines of code with no tests means every change is a gamble. A flaky bug surfaces weeks later, you don't know which commit caused it, and you're afraid to touch anything. Evals are the test suite for prompts. A skill that classifies commits well today might silently regress when you "improve" it tomorrow—evals catch that before it ships.

How It Works

1. **Build a 20–100 case dataset.** Include typical cases (60%), ambiguous cases (20%), and adversarial cases (20%). Store as JSONL:

```
{"id":"t01","input":"...", "expected":"...", "tag":"typical"}
```
2. **Run the prompt on each input.** Loop through the dataset, call the model with the prompt and input, collect the output.
3. **Grade each output.** If the answer is deterministic (classification, JSON shape), use code-based grading. If subjective (quality, thoroughness), use a Claude call as judge with temperature=0 and a strict rubric.
4. **Aggregate the scores.** Compute overall pass rate and per-tag rates. Wire the threshold into a pre-commit hook or GitHub Actions: fail if score drops below X or if any tag drops too far.

Pseudocode

```
dataset = load JSONL with input/expected pairs
results = []

for each case in dataset:
    actual_output = call_model(prompt, case.input)

    if classification (fixed label set):
        score = (actual == expected) ? 1.0 : 0.0
    else if subjective:
        score = llm_judge(actual, expected, rubric) returns 0..1

    record {case_id, tag, score}
    results.append(record)

overall = mean of all scores
by_tag = group by tag, compute mean for each

if overall >= 0.85 and all tags >= 0.6:
    print PASS
    exit 0
else:
    print FAIL with breakdown
    exit 1
```

Common Misconceptions

- **Myth:** "Evals are only for research; too slow for iteration."
Reality: 50 cases on Sonnet costs ~\$0.04. CI runs are cheap. Sample per-PR, run full on main.
- **Myth:** "I can use Claude to generate test cases AND Claude to grade them."
Reality: You're measuring "Claude agrees with itself"—useless. Either hand-label the dataset or use code-based grading.
- **Myth:** "If overall score passes, I'm safe."
Reality: Aggregate scores hide regressions in edge cases. Add per-tag floors: "no tag may drop below 0.6."

Key Takeaway

Evals turn prompt engineering from folklore into testable science. A 50-line eval runner + 20 test cases, wired into CI, costs almost nothing and catches regressions before they ship. Code-based grading (exact match, regex, JSON shape) is free and sufficient for classification; model-based grading (LLM-as-judge with temperature=0) handles subjective outputs. Always validate the judge against human scores before trusting it.

Quick Quiz

Q1. You're grading a "feat/fix/chore/docs/refactor?" classifier. Best grader?

- A) Code-based exact-match—answer is one of 5 fixed strings.
- B) LLM-as-judge with a rubric.
- C) Human review only.
- D) Doesn't matter.

Answer: A — Classification with a small fixed label set is the canonical case for code-based grading. It's free, deterministic, and fast. Save LLM-as-judge for subjective outputs.

Q2. Your model-based judge gives score 4.5 one run, 3.0 the next, on the same input. Why?

- A) The candidate output changed.
- B) You forgot `temperature=0` on the judge call.
- C) Anthropic released a new model.
- D) Cache miss.

Answer: B — Judges must be deterministic. Always set `temperature=0`, force the score via tool use, and use a stable model ID. Score variance run-to-run signals a non-zero temperature.

Q3. Your eval passes overall (0.88) but "adversarial" drops from 0.7 to 0.4. Ship?

- A) Yes—overall passed.
- B) No—adversarial drop signals overfit; add per-tag floor to gate.
- C) Yes—adversarial cases don't matter.
- D) Re-run; might be noise.

Answer: B — Aggregate scores hide regressions in important slices. A 30-point drop on one tag is a strong overfit signal. Always include a per-tag floor (e.g., "no tag below 0.6") and treat sharp drops as red flags.

RAG for Claude Code

Retrieval, embeddings, BM25, hybrid pipelines—and how to wrap RAG as a tool.

The Big Idea

RAG (Retrieval Augmented Generation) solves the "too much knowledge" problem: instead of fitting your entire codebase or doc library into Claude's context (expensive, slow, bounded), you retrieve only the relevant chunks at query time and pass them to Claude. RAG has two phases: retrieval (find matching passages) and generation (Claude reads them and answers). Use RAG when your knowledge base exceeds context length, changes frequently, or requires citations. Skip it if you have under 100K tokens—a cached system prompt wins.

Analogy

A new engineer joins your team. Do you make them memorize the entire codebase, or give them grep and let them search? Option one is impossible; option two is how everyone works. Claude is that engineer. RAG is the grep.

How It Works

1. **Chunk:** Split your documents into passages (fixed-size with overlap, semantic chunks, or code-aware via AST).
2. **Embed:** Vectorize each chunk using an embedding model (Voyage, OpenAI, Sentence-Transformers, Cohere).
3. **Index:** Store vectors in a vector database and build a BM25 index for keyword search.
4. **Retrieve:** At query time, run BM25 and vector search in parallel, merge results via Reciprocal Rank Fusion, optionally rerank with a cross-encoder.
5. **Augment & Generate:** Insert top-K passages into the prompt; Claude answers with grounding and citations (CC10).

Pseudocode

```
INGEST PHASE (offline)
  for each document:
    split into chunks with overlap
    embed each chunk → vector
    store vector + metadata (source, title)
  build BM25 index on tokenized chunks

QUERY PHASE (per request)
  tokenize user query
  bm25_results = BM25 search for top 50 chunks
  vector_results = vector similarity search for top 50
  merged_results = combine via reciprocal rank fusion
  top_k = rerank merged_results if using cross-encoder
  prompt = insert top_k chunks as context blocks
  answer = Claude reads chunks and responds with citations
```

Common Misconceptions

- **Myth:** Embeddings alone are enough. **Reality:** Embeddings excel at semantic similarity but fail on exact identifiers (error codes, class names, acronyms). Hybrid retrieval (BM25 + vectors) is the production standard.
- **Myth:** More retrieval stages = better accuracy. **Reality:** Each stage adds latency and failure points. Only add complexity (reranking, multi-stage filtering) if your eval (CC11) shows measured improvement.
- **Myth:** Store chunks without metadata. **Reality:** Metadata (source path, title, byte range) must be attached at ingest time. Without it, Claude cannot generate correct citations.

Key Takeaway

RAG is just another tool in Claude Code's loop (CC8)—instead of making Claude memorize, let it search. For CLI devs, wrap your retrieval as an MCP tool or subagent, and Claude automatically decides when to call it based on the task. Hybrid BM25 + vector retrieval covers both exact matches and semantic meaning; add reranking only if your eval proves it helps.

Quick Quiz

Q1. Your knowledge base is 50K tokens. Build a RAG pipeline?

- A) Yes, that's huge.
- B) No—50K fits in a cached system prompt; skip RAG.
- C) Yes, but only if you reindex weekly.
- D) Doesn't matter.

Answer: B — 50K tokens fits comfortably in context with prompt caching (CC10). RAG's value appears when the corpus exceeds context length or changes frequently. At 50K tokens with cached context, RAG adds engineering for no win.

Q2. Users search by error codes like "ECONNREFUSED". Use embeddings only, BM25 only, or hybrid?

- A) Embeddings only—semantic search is modern.
- B) Hybrid (BM25 + vectors)—exact identifiers need lexical precision.
- C) BM25 only is enough.
- D) Doesn't matter.

Answer: B — Error codes, class names, and acronyms favor exact-match retrieval. BM25 finds the literal token; embeddings cluster by meaning, which sometimes misses. Hybrid retrieval gets both precision and recall.

Q3. Your RAG answers are accurate but the citations don't match the actual sources. What's wrong?

- A) The model hallucinated.
- B) You didn't store source metadata with chunks at ingest.
- C) Embeddings are low quality.
- D) The reranker dropped the right chunk.

Answer: B — Metadata (source path, title, byte range) must be attached at ingest time. If the prompt doesn't include "this chunk came from refunds-v3.md," Claude has no choice but to invent a citation. Always store metadata; use document blocks in the prompt for guaranteed accurate citations (CC10).

CC13: Workflows, Agents & Computer Use

Four canonical patterns for orchestrating multi-step Claude work plus computer use for GUI automation.

The Big Idea

Most teams reach for "an agent" when "a workflow" would be cheaper, faster, and more reliable. Workflows are deterministic orchestrations you control; agents adapt but cost unpredictably. Four canonical patterns exist: chaining (sequential calls), parallelization (fan-out work), routing (cheap classifier picks specialist), and the agent loop (model decides steps). Plus computer use: Claude can screenshot desktops and click/type, useful for legacy GUIs without APIs but expensive and slow compared to direct APIs. Learn when each pattern wins.

Analogy

A workflow is a recipe: step 1, step 2, step 3, done. Anyone can follow it. The recipe doesn't change based on whether the milk smells funny—you just follow the steps. An agent is a chef. Given a goal and a kitchen, they decide what to cook, taste as they go, change course if things go wrong, and stop when the meal is ready. Both are useful. Recipes are predictable and cheap. Chefs handle novel situations and partial information.

How It Works

1. **Chaining:** Call A's output is Call B's input. Use for decomposable tasks—summarize, classify, route. Each step is smaller, cheaper, debuggable.
2. **Parallelization:** Fan out identical or independent work. Sectioning splits the work; voting runs the same task N times and takes majority. Use semaphores to cap concurrency; use Batch API for large offline jobs.
3. **Routing:** One cheap call classifies the request, then dispatches to a specialized handler. Avoids bloating every prompt with every system instruction.
4. **Agent Loop:** Tool-use loop where Claude picks tools, observes results, decides next step. Needs three caps: max iterations, max cost, max wallclock time—without them, buggy tools cause runaway bills.
5. **Computer Use:** Claude takes screenshots, clicks, types. For legacy GUIs with no API. Always sandbox in a VM. Slower and more expensive than API calls.

Pseudocode

```
WORKFLOW chaining(task, models):
    result_a = call_model(models.haiku, "summarize", task)
    result_b = call_model(models.sonnet, "classify", result_a)
    result_c = call_model(models.sonnet, "route", result_b, tools)
    return result_c

WORKFLOW agent_loop(goal, tools, max_iters, max_cost):
    history = [user: goal]
    spent = 0
    for iter in 1..max_iters:
        response = call_model("sonnet", history, tools)
        history.append(assistant: response)
        spent += cost(response)
        if spent > max_cost: return "BUDGET EXCEEDED"
        if response.stop_reason == "end_turn": return response.text
        for each tool_call in response.tool_uses:
            result = execute_tool(tool_call)
            history.append(user: tool_result)
    return "MAX ITERATIONS"

WORKFLOW computer_use_task(goal):
    loop:
        screenshot = take_screenshot()
        action = claude_vision(screenshot, goal) // returns {action,
coordinates, text}
        execute_action(action)
        if goal_complete(): break
```

Common Misconceptions

- **Myth:** "Agents are always smarter." **Reality:** Agents are more adaptive but predictably looping and opaque. For known steps, workflows are faster and cheaper. Most teams over-use agents.
- **Myth:** "Computer use is great for web automation." **Reality:** Computer use is slow (many screenshot/action rounds) and expensive (large image inputs). If a site has an API or Selenium works, use those instead.
- **Myth:** "I can run an unbounded agent loop and trust it to stop." **Reality:** A single buggy tool returning "try again" loops forever, billing unlimited amounts. Always set max iterations, max cost, and max time.

Key Takeaway

Default to workflows—they're cheaper, faster, and debuggable. Reserve agents for genuinely open-ended tasks where steps can't be enumerated upfront. When you do use agents, cap all three: iterations, cost, and time. Computer use is for legacy GUIs only; prefer APIs.

Quick Quiz

Q1. You can decompose your task into 3 fixed steps. Should you build an agent or workflow?

- A) Agent—agents are more capable.
- B) Workflow—deterministic structure means cheaper, faster, easier to debug.
- C) Doesn't matter.
- D) Both, in parallel.

Answer: B — If you know the steps, encode them in code, not in the model. You'll get predictable cost, easier debugging, and better evals (each stage tested independently).

Q2. Your agent loop hangs occasionally. What did you forget?

- A) To set max_tokens.
- B) Iteration / cost / time caps. A buggy tool can spin forever.
- C) To enable streaming.
- D) To use Opus.

Answer: B — Production agents need bounded loops: max iterations, max cost, max wallclock. Without all three, a tool returning "try again" or a misbehaving model can rack up unbounded charges.

Q3. You want Claude to fill in a form on a legacy desktop app with no API. What's appropriate?

- A) Build a chain workflow.
- B) Computer use, in a sandboxed VM, with iteration + cost caps.
- C) RAG.
- D) Computer use without a sandbox—just trust Claude.

Answer: B — Legacy GUI with no API is the textbook computer-use case. Always sandbox it (security) and cap iterations (cost)—computer use is the easiest way to spend a lot of tokens fast if the model gets confused.

CC14: Power User & CI/CD

Master extended thinking, parallel sessions, and Claude-driven code review in production pipelines.

The Big Idea

Advanced Claude Code workflows live in three layers: deeper reasoning (extended thinking), session management (branching, worktrees, resumption), and CI/CD integration (headless CLI, GitHub Actions). Power users combine these tools to ship code faster with fewer bugs. Extended thinking handles complex tasks by letting Claude reason longer. Worktrees let you run multiple sessions on the same repo without conflicts. The two-Claude review pattern shifts left by having one session generate and another review in isolation, catching bugs before they hit main.

Analogy

Extended thinking is like taking a hard problem home to sleep on it—your subconscious works through the complexity before you respond. Worktrees are like having multiple desks: you can sketch out idea A at one desk and idea B at another without papers flying everywhere. The two-Claude pattern is code review without the ego: because the reviewer session has no context of why the author made their choices, it sees logic gaps the author can't.

How It Works

1. **Trigger extended thinking five ways:** settings.json (alwaysThinkingEnabled), per-session (/effort high|max), inline (include "ultrathink"), per-skill (effort: max), or UX toggle (Option+T / Alt+T). Control budget via MAX_THINKING_TOKENS env var.
2. **Fork sessions without losing state:** Use /branch to diverge at your current point, storing two timelines in one session. Pair with --worktree for separate file systems. Resume work with --continue, --resume, or --resume session-id.
3. **Two-Claude review:** Session A generates code non-interactively (claude -p). Session B reviews output with zero context of the generation reasoning, catching assumptions author missed.
4. **Run headless in CI:** Use claude -p (non-interactive), --output-format json, --allowedTools (whitelist only what CI needs), --permission-mode acceptEdits, --max-turns (bound the loop), and --append-system-prompt to constrain output shape.

Pseudocode

```
// Extended thinking: trigger and budget
IF task is complex THEN
  set Option+T (UX toggle) OR include "ultrathink" in prompt
  environment MAX_THINKING_TOKENS = 10000
END

// Session management: branch and worktree
COMMAND /branch "attempt B" -- fork timeline without losing context
COMMAND --worktree=feature-2 -- separate file system for parallel work
COMMAND --resume=session-xyz -- restore stopped session

// Two-Claude review
SESSION_A: claude -p "generate code" --output-format json > output.json
SESSION_B: read output.json, review for gaps, logic errors, assumptions
CI_OUTPUT: structured JSON from JSON output, drive next step

// GitHub Actions: headless Claude Code
RUN claude -p "review PR" \
  --output-format json \
  --allowedTools "Read,Grep,Bash(git diff:*)" \
  --permission-mode acceptEdits \
  --max-turns 10 \
  --append-system-prompt "Return ONLY JSON array of issues"
POST RESULT to GitHub PR comment
```

Common Misconceptions

- **Myth:** Extended thinking only works for reasoning tasks. **Reality:** It improves output quality on any task requiring careful thought—refactoring, API design, security review—not just math and logic.
- **Myth:** One Claude session can review its own code with a "review mode" flag. **Reality:** The reviewing turn inherits the generation reasoning in context, biasing it toward agreement. A second session sees the code with fresh eyes, catching real bugs.
- **Myth:** Worktrees need manual .env setup. **Reality:** Use .worktreeinclude in settings.json to symlink .env.local, .secrets, and other dev files into new worktrees automatically.

Key Takeaway

Power users ship code faster by layering extended thinking (reasoning budget), session management (parallel exploration), and two-Claude review (second pair of eyes). In CI, the headless CLI (-p, --output-format json, tight tool allowlists) keeps Claude Code sandboxed and predictable. Together, these patterns reduce the feedback loop from hours to minutes.

Quick Quiz

Q1. Multiple approaches to a refactor without losing state—which primitive?

- A) Open a second terminal and run claude there.
- B) claude --resume with a new approach.
- C) /branch to fork the session into two timelines.
- D) /clear and start over.

Answer: C — /branch forks at the current point into two timelines. Pair with --worktree for separate file systems. Two terminals would race on files; --resume loses the diverging-from-this-point structure.

Q2. CI needs to call Claude Code once and get structured JSON. Which flags?

- A) claude --interactive --json
- B) claude -p "prompt" --output-format json --permission-mode acceptEdits --allowedTools "Read,Grep,Bash(git diff:*)" --max-turns 15
- C) claude --resume CI --headless
- D) claude --watch --json-out

Answer: B — Headless CI requires -p (non-interactive), --output-format json (structured output), --allowedTools (sandboxed tools), --permission-mode acceptEdits (auto-approve safe edits), and --max-turns (bound the loop). Use --append-system-prompt to constrain output shape.

Q3. A worktree starts fresh without .env.local, so the dev server fails. Cleanest fix?

- A) git add .env.local; commit it (exposes secrets).
- B) Add .env.local to .worktreeinclude.
- C) Set worktree.symlinkDirectories in settings.json.
- D) Manually cp .env.local to worktree each time.

Answer: B — .worktreeinclude symlinks specified files into new worktrees automatically, keeping secrets out of git. Files listed in .worktreeinclude are never committed, and every new worktree gets the symlink. .gitignore files by default; .worktreeinclude exempts them from that rule per worktree.

CC15: Building Custom Agents with the Claude Agent SDK

Package Claude Code's agent engine as a Python and TypeScript library for background jobs, scheduled tasks, and programmatic automation.

The Big Idea

The first 14 modules of the CC track taught you how to use Claude Code interactively at the terminal. But most real agent problems run unattended: a GitHub Action that reviews pull requests, a Slack bot triaging support tickets, a nightly audit job, or a customer-facing chat surface answering questions. The Claude Agent SDK gives you Claude Code's full capabilities—tools, file system access, MCP servers, subagents, hooks—but without the interactive terminal. It's the same engine packaged as a Python and TypeScript library, so you can bake agent logic directly into your applications.

Analogy

Think of Claude Code as a powerful manual transmission sports car: you sit in the driver's seat, steer, shift gears, and feel every control. The Agent SDK is the same car's engine sold separately, ready to power a custom vehicle of your design. Same power, same reliability, different interface. You set the course, define what tools are allowed, cap the turns, and let it run autonomously while you attend to other work.

How It Works

1. **Import and configure:** Create a `ClaudeAgentOptions` object (or TypeScript equivalent) specifying your system prompt, allowed tools, MCP servers, max turns, and permission mode.
2. **Define your tools:** Use built-in tools (Read, Write, Edit, Bash, Grep, Glob) or wrap your own REST APIs as MCP servers decorated with `@tool`, then point the SDK at them via `mcp_servers`.
3. **Call `query()`:** Pass your prompt and options to the `query()` function; it returns an async stream of messages: `SystemMessage` (setup), `AssistantMessage` (Claude's turns with tool calls), `UserMessage` (tool results), and `ResultMessage` (final metrics).
4. **Stream and inspect:** Iterate through the message stream, handling text blocks, tool calls, and results. Log, validate, or conditionally handle each step of the agent loop.
5. **Deploy:** Run in Docker, Lambda, Cloud Functions, cron jobs, or any Python/TypeScript runtime. The SDK handles tool dispatch, MCP integration, token budgets, and multi-turn state—no manual orchestration needed.

Pseudocode

```
// Define the agent once
options = {
    system_prompt: "You are a UCC filing assistant...",
    allowed_tools: ["mcp__ucc__lookup_filings", "mcp__ucc__get_filing"],
    mcp_servers: [create_sdk_mcp_server(ucc_tools)],
    max_turns: 5,
    permission_mode: "bypassPermissions" // CI mode: no prompts
}

// Run the agent once
async for message in query(user_prompt, options):
    if message is SystemMessage:
        log("Session started", message.session_id)

    else if message is AssistantMessage:
        for block in message.content:
            if block is TextBlock:
                print(block.text)
            else if block is ToolUseBlock:
                log("Calling", block.name, "with", block.input)

    else if message is UserMessage:
        log("Tool returned", message.content[0].result)

    else if message is ResultMessage:
        log("Complete in", message.duration_ms, "ms,",
            message.total_cost_usd, "cost,", message.num_turns, "turns")

// Multi-turn variant (for interactive or persistent sessions)
client = ClaudeSDKClient(options)
for each user query:
    response = client.query(user_query) // continues conversation
    handle response as above
```

Common Misconceptions

- **Myth:** The Agent SDK uses a different Claude model than the Messages API. **Reality:** Same model. The SDK's value is the scaffolding (tool loop, MCP integration, built-in Read/Write/Bash, sessions) so you don't hand-roll it. Pick Messages API only when you need full custom control.
- **Myth:** I must always use `allowed_tools` to restrict tools; it's required. **Reality:** It's optional but strongly recommended. Without it, your agent could call Bash, Edit, or Write—capabilities a read-only data-fetching agent doesn't need. Defense in depth: limit the agent's surface to only what its job requires.
- **Myth:** The Agent SDK is only for Python; TypeScript isn't well-supported. **Reality:** TypeScript support is full-featured and identical. Both languages have the same `query()` API, `ClaudeSDKClient` class, message types, and MCP integration. Choose by what your

deployment target runs.

Key Takeaway

The Agent SDK is Claude Code's engine separated from its terminal interface. Use it when you're building unattended agents—background jobs, APIs, chat bots, automation—that need tool use, file access, and multi-turn reasoning without human interaction. It's the sweet spot between the CLI (most opinionated) and the Messages API (most flexible). For most "I want an agent that does X" problems, the SDK is your starting point.

Quick Quiz

Q1. Your team needs a nightly cron job that audits a Postgres warehouse for new UCC filings and posts to Slack. Which Claude surface?

- A) Claude Code CLI—just run "claude -p audit and post" in cron
- B) Claude Agent SDK—a Python program with Postgres + Slack MCP servers
- C) Anthropic Messages API—write the tool loop and dispatch yourself

Answer: B — Background jobs that need tools, MCP, and an agent loop without a terminal are the SDK's sweet spot. The CLI works for interactive pair-programming but you lose programmatic control. The Messages API gives you maximum flexibility but requires hand-rolling the loop and retries.

Q2. In the lab, what's the role of `create_sdk_mcp_server` / `createSdkMcpServer`?

- A) It launches the Claude Code interactive UI in a child process
- B) It packages your tool definitions into an in-process MCP server the SDK loads
- C) It registers the agent with the Anthropic API for remote invocation

Answer: B — The function returns an MCP server you wire into `options.mcp_servers`. The SDK runs it in-process; there's no separate subprocess. It's the bridge between your `@tool`-decorated functions and the SDK's MCP loader.

Q3. `allowed_tools=["mcp_ucc_lookup_filings", "mcp_ucc_get_filing"]`. Why this list instead of `None` (allow everything)?

- A) The SDK requires an explicit list to function
- B) Defense in depth—without a list, the agent could call Bash/Edit/Write capabilities it doesn't need
- C) It's required for the MCP server to load correctly

Answer: B — This is the principle of least privilege. A read-only data agent has no business running shell commands or editing files. An `allowed_tools` list ensures the agent can do its job and nothing else, just like CC4's permission tiers and CC6's read-only auditor subagent.